Introduktion til HTTP

Opgavebeskrivelse

Selvom HTTP i dag stort set altid anvendes i sin krypterede form som *HTTPS*, er det fortsat vigtigt at have en grundlæggende forståelse for den oprindelige HTTP-protokol. HTTPS er i praksis blot HTTP kombineret med et krypteringslag *(TLS)*, og mange sikkerhedsmæssige udfordringer opstår netop i HTTP-laget – før data overhovedet krypteres. For at kunne analysere og forstå sårbarheder som eksempelvis *header-manipulation, session hijacking* og *cookie-tyveri*, er det derfor nødvendigt at kende til, hvordan HTTP fungerer. En solid forståelse af HTTP giver dermed et bedre grundlag for at vurdere og styrke webapplikationers sikkerhed, også når de kommunikerer over HTTPS

Den grundlæggende protokol for overførelse af data, dvs. tekst, billeder osv., på *the World Wide Web* kaldes *Hypertext Transfer Protocol* og forkortes til HTTP. Ordet Hypertext kendes også fra *Hybertext Markup Language (HTML)*. Det særlige ved hypertekst fremfor almindelig tekst er, at dokumenterne er kædet sammen med hyperreferencer, dvs. *links*. En webserver er en tjeneste, som stiller information til rådighed ved brug af HTTP. Webserveren er et program til anvendelse, dvs. hvad vi også kalder en applikation. HTTP er det sprog, som webserveren og brugerens computer bruger til at kommunikere. Helt i tråd med OSI-modellen, kender applikationen kun til laget umiddelbart under

---

Afleveringsformat

Til denne uges opgave skal I lave et Github-repo, der indeholder følgende:

- En README.md med kort beskrivelse af opgaver og programmer:
    - README'en skal fungere som en logbog over den forståelse du opnår, og de refleksioner du gør dig omkring hver opgave, fx:
        - Hvad lærte du om HTTP i den pågældende øvelse, og hvorfor er det relevant for cybersikkerhed?
    - Beskrivelser og instruktioner i at køre de mindre ad hoc-programmer fra opgaverne
- Mapper med kodeopgaver og evt. øvrige filer.

Øvelser

Vi har udarbejdet nogle simple øvelser, som vi anbefaler at man gennemgår manuelt. De skulle gerne give et førstehåndsindtryk af HTTP. Det er vort håb at formidle, at HTTP i sig selv er ret simpel.

Vi vil i denne opgave bruge programmet `curl` til at kommunikere med en webserver. For at holde selve øvelsen enkel har vi to indledende trin. Der skrives en HTML-fil, og denne udstilles ved hjælp af en webserver. På den måde har vi fuld kontrol over klient (cURL), server (webserveren) og indholdet (HTML-filen). Det ønsker vi for at kunne se og forstå, hvad der sker.

Opret først en HTML-fil. Følgende eksempel `index.html` er fin til formålet.

```
<!doctype html>
<html lang="da-DK">
    <head>
        <meta charset="utf-8">
        <title>SPAC Cyber</title>
    </head>
    <body>
        <h1>Enkel webside</h1>
        <p>Prøv fx følgende:</p>
        <ul>
            <li>Åbn siden i en webbrowser</li>
            <li>Hent siden med cURL</li>
            <li>Hent siden med Netcat</li>
        </ul>
    </body>
</html>
```

Det antages i det følgende, at læseren har adgang til en computer, hvorpå Python 3 er installeret og kan køres med `python`.

Vi vil gerne kunne skrive i ét terminalvindue samtidig med, at vi også kan følge logningen i et andet. Man kan eventuelt anvende en virtuel terminal, som kan holde flere tabs åbne ved siden af hinanden, eller bruge en *terminal multiplexer* fx `screen`, `tmux` eller `zellij`.

Python er berømt for at være *batteries included*, og der medfølger da også en simpel webserver i form af et modul. I mappen hvor HTML-filen blev gemt køres ganske enkelt:

```
python -m http.server
```

Åbn eventuelt http://localhost:8000/index.html i en webbrowser for at kontrollere, at alt virker. Det kan være en idé at dræbe webserveren og starte den igen herefter for at rydde loggen.

Øvelse 1: HTTP GET med cURL

Åbn nu endnu en kommandoliniefortolker ved siden af den, hvori webserveren blev startet. På den måde kan man følge serverens logning undervejs. Brug nu cURL til at udføre et HTTP GET request til webserveren. Det gøres enkelt ved at køre:

```
curl http://localhost:8000/index.html
```

HTML-filen skulle gerne udskrives til skærmen uændret.

Øvelse 2: cURL med headers

Den body som blev vist i eksemplet herover er ikke hele det svar, som webserveren sender. Et svar består af en statuslinie, nogle linier med informationer kaldet *headers* og endelig ofte også et dobbelt linieskifte efterfulgt af en body som i vort eksempel. Lav en ny forespørgsel med cURL, hvor cURL får instruktioner om også at printe statuslinien samt headers.

```
curl -i http://localhost:8000/index.html
```

Det skulle gerne resultere i noget, som ligner følgende, men tidspunkter mv. vil naturligvis være anderledes.

```
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.13.7
Date: Wed, 08 Oct 2025 11:14:22 GMT
Content-type: text/html
Content-Length: 380
Last-Modified: Wed, 08 Oct 2025 11:01:40 GMT

<!doctype html>
<html lang="da-DK">
    <head>
```

```
        <meta charset="utf-8">
        <title>SPAC Cyber</title>
    </head>
    <body>
        <h1>Enkel webside</h1>
        <p>Prøv fx følgende:</p>
        <ul>
            <li>Åbn siden i en webbrowser</li>
            <li>Hent siden med cURL</li>
            <li>Hent siden med Netcat</li>
        </ul>
    </body>
</html>
```

Det er det rå HTTP-svar. HTTP er en tekstbaseret protokol.

Øvelse 3: Verbose cURL

Kør cURL med instruks om omfattende output (verbose)

```
curl -v http://localhost:8000/index.html
```

Herunder gives et eksempel på outputtet i terminalen:

```
➜  ~ curl -v http://localhost:8000/index.html
* Host localhost:8000 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8000...
* connect to ::1 port 8000 from ::1 port 51478 failed: Connection refused
*   Trying 127.0.0.1:8000...
* Connected to localhost (127.0.0.1) port 8000
* using HTTP/1.x
> GET /index.html HTTP/1.1
> Host: localhost:8000
> User-Agent: curl/8.11.1
```

```
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.13.7
< Date: Wed, 15 Oct 2025 11:59:40 GMT
< Content-type: text/html
< Content-Length: 377
< Last-Modified: Wed, 08 Oct 2025 12:30:25 GMT
<
<!doctype html>
<html lang="da-DK">
    <head>
        <meta charset="utf-8">
        <title>SPAC Cyber</title>
    </head>
    <body>
        <h1>SPAC Cyber</h1>
        <p>Prøv fx følgende:</p>
        <ul>
            <li>Åbn siden i en webbrowser</li>
            <li>Hent siden med cURL</li>
            <li>Hent siden med Netcat</li>
        </ul>
    </body>
</html>
* shutting down connection #0
➜  ~
```

Af outputtet fremgår nu en række skridt, som ligger før HTTP. Før der kan udveksles HTTP-beskeder, skal der oprettes en TCP-forbindelse. Først bestemmes serverens IP-adresse. Der vil typisk gives to: én for IPv4 og én for IPv6. Vor HTTP-server lytter alene på IPv4, så et eventuelt indledende forsøg på at oprette forbindelse til IPv6-adressaten vil fejle. Linierne omhandlende forberedelsen før HTTP vil af cURL indledes med *. De følgende linier indledt med > er den rå data sendt over TCP-

forbindelsen. Der vil være en forespørgselslinie (en. *request line*) efterfulgt af en række headerlinier (her typisk tre). Forespørgslen afsluttet med en tom linie (teknisk: et ekstra linieskift efter den sidste header). HTTP er en tekstbaseret protokol, så *både forespørgselslinien og headerlinierne er læsbar ASCII-teksk*.

Efter forespørgselslinien følger cURLs egne beskeder med foranstillet `*`. De derpå følgende linier med foranstillet `<` er det rå HTTP-svar. Svaret består af en statuslinie, en række headerlinier, en tom linie og endelig selve beskedeindholdet, som i dette tilfælde er vor `index.html`. Beskedeindholdet gengives verbatimt, og foranstilles altså ikke med `<`.

Outputtet afsluttes kort med en besked fra cURL om, at TCP-forbindelsen er blevet lukket igen.

Øvelse 4: Manuel HTTP-forespørgsel med netcat

I øvelsen med cURL så vi rå HTTP-beskeder. Vi har en HTTP-server kørende (`python -m http.server`), så lad os prøve manuelt at sende den en HTTP-forespørgsel. HTTP er en tekstbaseret protokol oven på TCP, så vi skal bruge en TCP-forbindelse til serveren. Den kan vi oprette ved at bruge programmet *Netcat*.

Programfilen `netcat` kaldes ofte ved `nc`. Som mange andre utilities på Linux og Unix findes *Netcat* i flere udgaver, som afviger lidt fra hinanden. *Ncat* er et helt andet program med lignende funktionalitet. Ncat hører til *Nmap*.

Lad os åbne en rå TCP-forbindelse til webserveren.

```
netcat localhost 8000
```

Outputtet vil måske skuffe ved sit fravær og ligne eksemplet herunder: bemærk dog den blinde cursor i anden linje.

```
➜  ~ netcat localhost 8000
```

TCP-forbindelsen *er* oprettet. Hvad der herefter gives af input sendes uændret til webserveren. Lad os sende en HTTP-forespørgsel (request). Skriv følgende efterfulgt af *dobbelt* linjeskift

```
GET /index.html HTTP/1.1
```

Opgaver

Opgaverne er relateret til øvelserne. Opgavernes numre er ikke relateret til øvelsernes numre.

Opgave 1

Se på HTTP-forespørgslerne og svarene i øvelserne. I øvelserne kommunikerer vi blot internt på vores egen maskine, men vi kunne have kommunikeret med eksterne servere. Hvor i HTTP-forespørgslerne bestemmes adressen på serveren (domæne eller IP-adresse)?

Opgave 2

Hvordan forbryder vi os mod HTTP-standarden i øvelsen med Netcat?

Opgave 3

Lav en forespørgsel og bestem den præcise størrelse af svaret i bytes. Der er her tale om hele svaret, dvs. inkl. forespørgselslinje og headers. Lav og inspicér samme forespørgsel i en webbrowser og notér browserens bestemmelse af datastørrelsen. Sammenlign de to tal. Forklar formålet med denne opgave.

Opgave 4

Vi har lavet HTTP-forespørgsler med cURL og Netcat. Skriv et lille program i Python, der sender en HTTP GET-forespørgsel `localhost:8000/index.html` og viser svaret til brugeren.

Til formålet kan man importere `socket` fra standardbiblioteket. For TCP kan bruges

```
socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Opgave 5

Skriv et lille program i Python, der svarer på HTTP GET-forespørgsler til `localhost:8000/index.html`. Hold svaret simpelt.

Opgave 6 (valgfri)

Dan par med en anden akademist. Fordel rollerne server og klient mellem jer. Serveren fortæller klienten sin IP-adresse og bruger Netcat til at lytte på en TCP-port (fx 8000: `netcat -l 8000`). Klienten opretter forbindelse hertil og sender HTTP-forespørgsler. Serveren fortolker disse og sender HTTP-svar tilbage. Hold øje med at I følger HTTP-standarden.

Ressourcer

- HTTP - Wikipedia
- RFC 9110 HTTP Semantics
- RFC 9111 HTTP Caching
- RFC 9112 HTTP/1.1
- RFC 9113 HTTP/2
- RFC 9114 HTTP/3
- cURL webside
- cURL - Wikipedia
- cURL - GitHub